Dear ImGui 入門ガイド

Pleiades + Win32 + DirectX11 / 環境構築・基本ウィジェット・日本語対応・Win32 API 比較

この資料は、アップロードされた **main.cpp / platform_dx11.cpp / platform_dx11.h / ui.cpp / ui.h** を土台に、Pleiades 上で Dear ImGui を使い始めるための実用ガイドとして再構成したものです。

前回の PDF で不足していた dwmapi については、ヘッダとリンクライブラリの方を環境構築セクションへ追記しています。

1. 前提と全体像

想定環境：Windows / Pleiades(Eclipse 系) / MinGW 系ツールチェーン / Win32 ウィンドウ / DirectX11 / Dear ImGui

今回の構成では、**main.cpp** がアプリ全体の流れ、**platform_dx11.cpp** が Win32 + DX11 の土台、**ui.cpp** が RenderUI による UI 本体です。

- main.cpp：WinMain、ImGui 初期化、日本語フォント読み込み、メインループ
- platform_dx11.h / .cpp：CreateDeviceD3D、CreateRenderTarget、WndProc などの基盤処理
- ui.h / .cpp：RenderUI のみを置き、今後の画面追加先を明確にする

2. Pleiades での環境構築

1. プロジェクトを C++ の Windows アプリとして作成する。コンソールアプリでも開始可能だが、最終的には WinMain を使う。
2. Dear ImGui 本体をプロジェクトへ配置する。最低限必要なのは imgui.cpp / imgui_draw.cpp / imgui_tables.cpp / imgui_widgets.cpp / imgui_demo.cpp(任意) / imgui.h / imgui_internal.h。
3. backends フォルダから imgui_impl_win32.cpp / imgui_impl_win32.h / imgui_impl_dx11.cpp / imgui_impl_dx11.h を追加する。
4. 自作ファイルとして main.cpp / platform_dx11.cpp / platform_dx11.h / ui.cpp / ui.h を src 配下に置く。

2.1 インクルードパス

- プロジェクトの Properties → C/C++ Build → Settings → Compiler → Includes で、Dear ImGui のルートフォルダを追加する。
- backends を参照できるよう、imgui ルートに加えて backends も含む構成にする。

```
#include <windows.h>
#include <tchar.h>
#include <d3d11.h>
#include <dwmapi.h>      // 追加：前回 PDF で不足していたヘッダ

#include "imgui.h"
#include "backends/imgui_impl_win32.h"
#include "backends/imgui_impl_dx11.h"
```

```
#include "platform_dx11.h"
#include "ui.h"
```

2.2 リンクライブラリ

Pleiades の Linker 設定には、少なくとも次を追加する。

ライブラリ	用途
d3d11	DirectX11 デバイスとレンダリング
dxgi	スワップチェーン管理
dxguid	GUID 解決で必要になることがある
dwmapi	Desktop Window Manager 連。前回 PDF の不足分

環境によっては imm32 を追加すると日本語 IME 周りが安定しやすいことがあります。

2.3 ソースファイル構成

```
project/
├─ src/
│  ├─ main.cpp
│  ├─ platform_dx11.cpp
│  ├─ platform_dx11.h
│  ├─ ui.cpp
│  └─ ui.h
├─ imgui/
│  ├─ imgui.cpp
│  ├─ imgui_draw.cpp
│  ├─ imgui_tables.cpp
│  ├─ imgui_widgets.cpp
│  ├─ imgui.h
│  └─ backends/
│     ├─ imgui_impl_win32.cpp
│     ├─ imgui_impl_win32.h
│     ├─ imgui_impl_dx11.cpp
│     └─ imgui_impl_dx11.h
```

2.4 main.cpp での日本語フォント読み込み

アップロードされた main.cpp では、ImGui コンテキスト作成後にメイリオを読み込む構成になっている。

```
IMGUI_CHECKVERSION();
ImGui::CreateContext();

ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
ImGui::StyleColorsDark();

ImFontConfig font_config;
```

```
font_config.MergeMode = false;
const ImWchar* ranges = io.Fonts->GetGlyphRangesJapanese();

io.Fonts->AddFontFromFileTTF(
    "C:/Windows/Fonts/meiryo.ttc",
    18.0f,
    &font_config,
    ranges
);
```

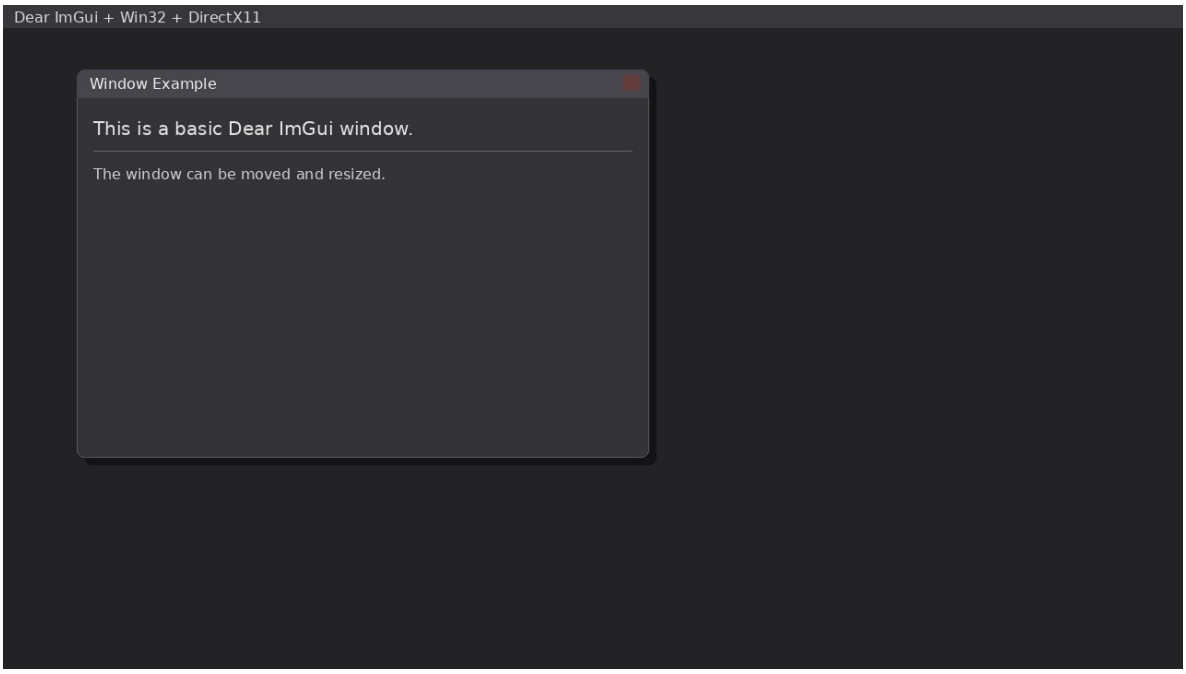
ソスコドは UTF-8 で保存する。日本語ラベルが化ける場合は、フォントパス・文字コード・GlyphRangesJapanese の 3 点を先に確認する。

3. ウィンドウ

- ImGui::Begin("タイトル") と ImGui::End() の間に UI を書く。
- SetNextWindowPos / Size は最初の表示位置・サイズ決めに便利。

```
void RenderUI()
{
    ImGui::SetNextWindowPos(ImVec2(40, 40), ImGuiCond_Once);
    ImGui::SetNextWindowSize(ImVec2(420, 180), ImGuiCond_Once);

    ImGui::Begin("Window Example");
    ImGui::Text("This is a basic Dear ImGui window.");
    ImGui::Separator();
    ImGui::Text("The window can be moved and resized.");
    ImGui::End();
}
```

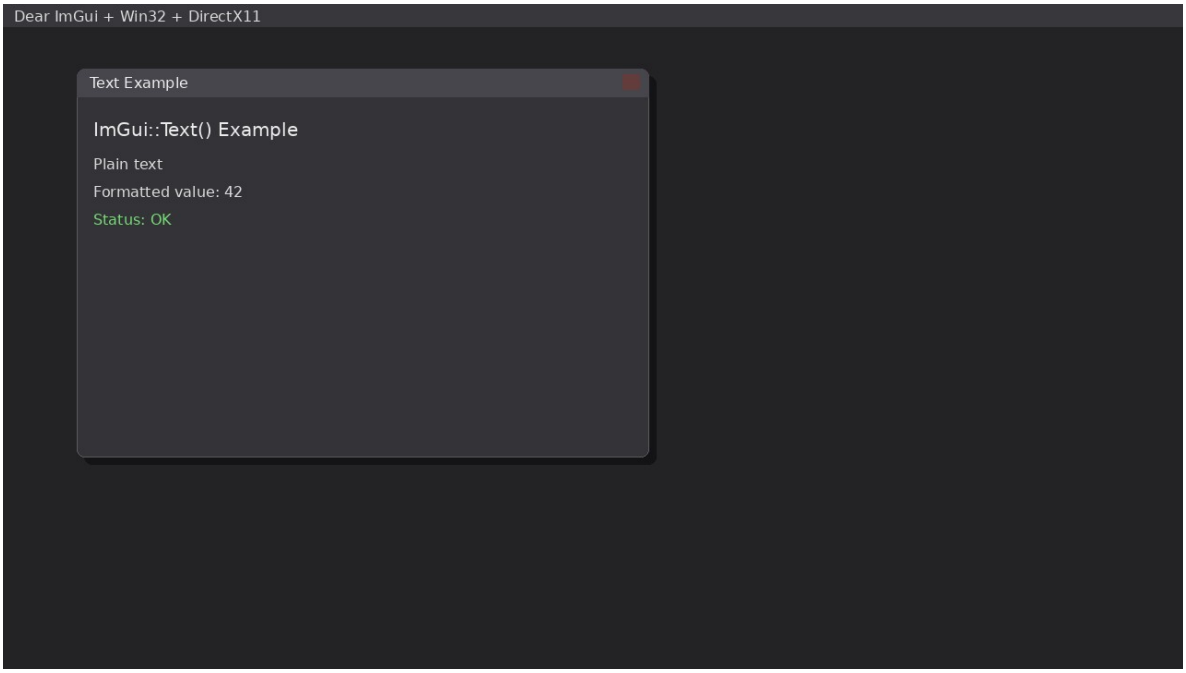


図：3. ウィンドウの画面イメージ

4. テキスト

- 固定文字列は ImGui::Text()、値の埋め込みは printf 風フォーマットで書ける。
- TextColored を使うと、ステータス表示だけ色を変える、といった表現が簡単。

```
void RenderUI()
{
    ImGui::Begin("Text Example");
    ImGui::Text("Plain text");
    ImGui::Text("Formatted value: %d", 42);
    ImGui::TextColored(ImVec4(0.4f, 0.9f, 0.4f, 1.0f), "Status: OK");
    ImGui::End();
}
```



図：4. テキストの画面イメージ

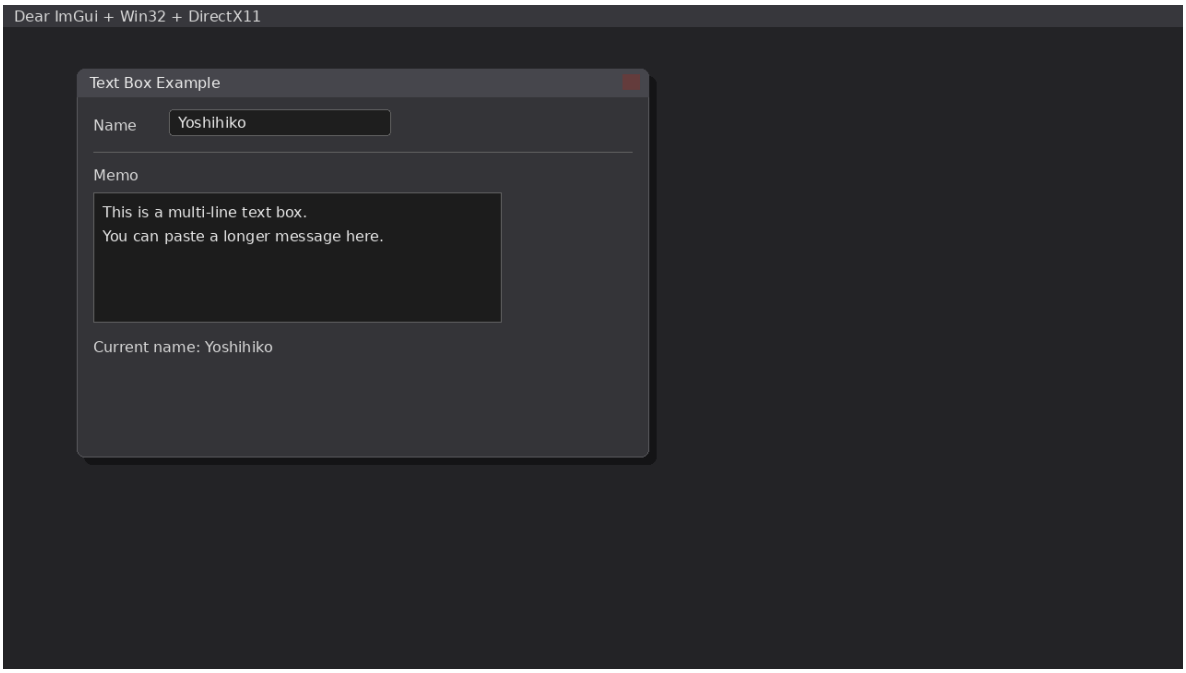
5. テキストボックス

- InputText / InputTextMultiline は std::string ではなく char 配列をそのまま渡すのが基本。
- 毎フレーム初期化されないよう、入力バッファは static かメンバ変数で保持する。

```
void RenderUI()
{
    static char input_name[128] = "Yoshihiko";
    static char memo[256] =
        "This is a multi-line text box.\n"
```

```
        "You can paste a longer message here.";

    ImGui::Begin("Text Box Example");
    ImGui::InputText("Name", input_name, IM_ARRAYSIZE(input_name));
    ImGui::Separator();
    ImGui::InputTextMultiline(
        "Memo",
        memo,
        IM_ARRAYSIZE(memo),
        ImVec2(440, 150)
    );
    ImGui::Text("Current name: %s", input_name);
    ImGui::End();
}
```



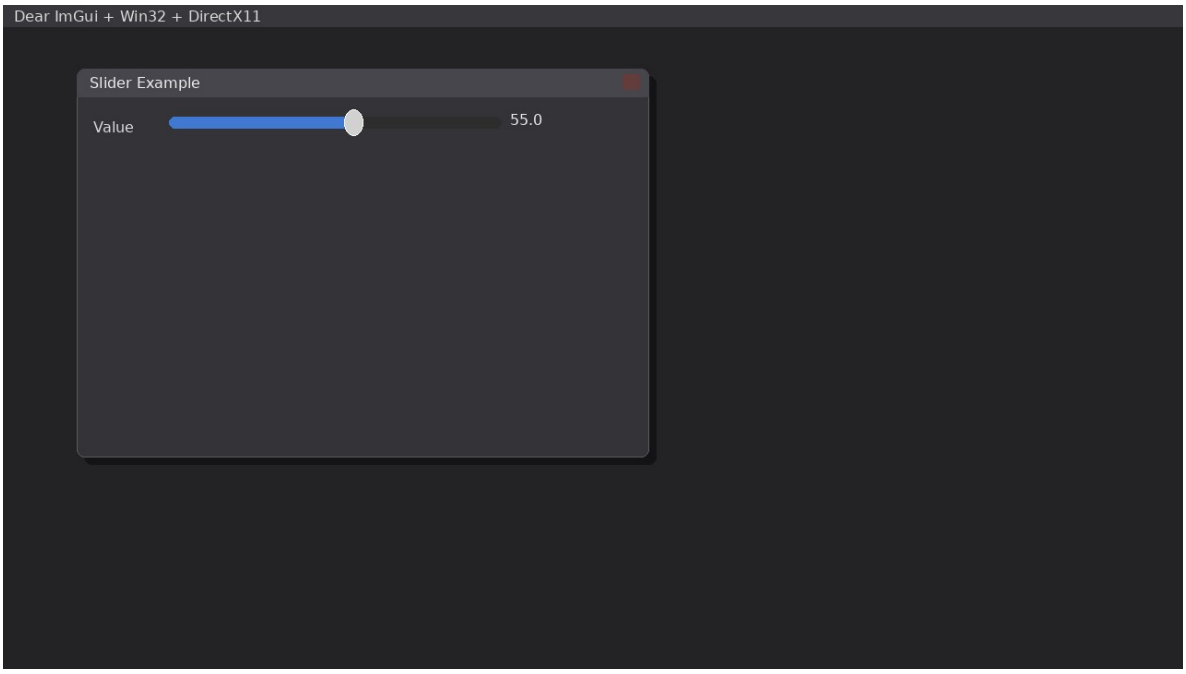
図：5. テキストボックス の画面イメージ

6. スライダー

- 数値を直感的に調整したいときは SliderFloat / SliderInt が便利。
- スライダーを動かすだけで状態変数が更新されるのが ImGui の分かりやすさ。

```
void RenderUI()
{
    static float value = 55.0f;

    ImGui::Begin("Slider Example");
    ImGui::SliderFloat("Value", &value, 0.0f, 100.0f);
    ImGui::Text("Current value = %.1f", value);
    ImGui::End();
}
```



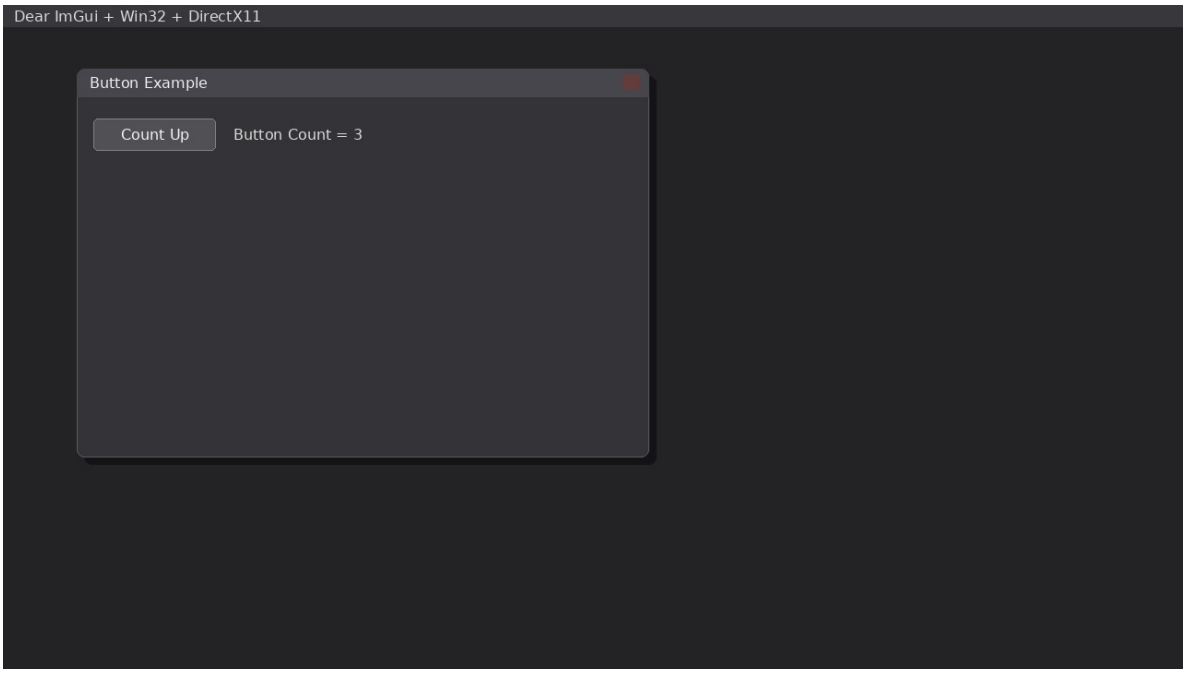
図：6. スライダー の画面イメージ

7. ボタン

- ボタンは押された瞬間だけ true を返す。
- SameLine() を使うと、ボタンの横に結果表示を並べられる。

```
void RenderUI()
{
    static int count = 0;

    ImGui::Begin("Button Example");
    if (ImGui::Button("Count Up"))
    {
        count++;
    }
    ImGui::SameLine();
    ImGui::Text("Button Count = %d", count);
    ImGui::End();
}
```



図：7. ボタンの画面イメージ

8. スクロール

- 大量の項目を 1 ウィンドウに詰め込むときは BeginChild が便利。
- 親ウィンドウとは別のスクロール領域として扱える。

```
void RenderUI()
{
    ImGui::Begin("Scroll Example");
    ImGui::Text("Scroll Area");

    ImGui::BeginChild("ScrollRegion", ImVec2(0, 180), true);
    for (int i = 0; i < 30; i++)
    {
        ImGui::Text("Scrollable Item %d", i + 1);
    }
    ImGui::EndChild();

    ImGui::End();
}
```

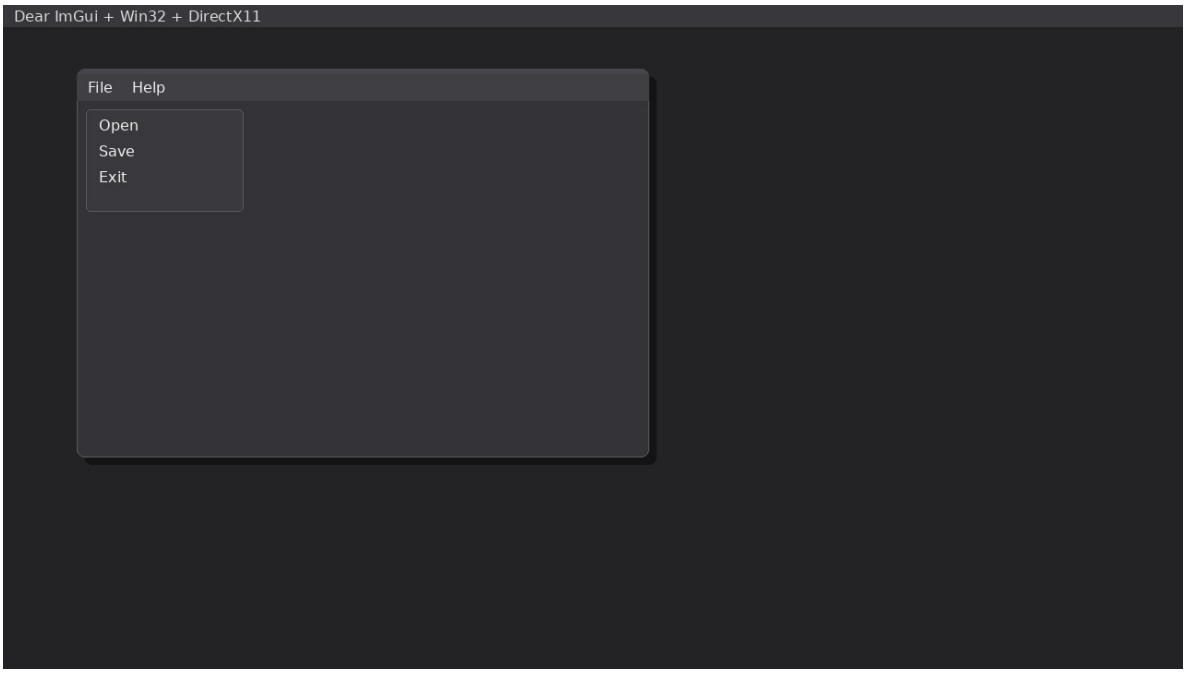
図：8. スクロール の画面イメージ

9. メニュー

- 画面上部のアプリ風メニューは BeginMainMenuBar で開始する。
- 実際の処理は MenuItem が true を返したときに書く。

```
void RenderUI()
{
    if (ImGui::BeginMainMenuBar())
    {
        if (ImGui::BeginMenu("File"))
        {
            ImGui::MenuItem("Open");
            ImGui::MenuItem("Save");
            ImGui::MenuItem("Exit");
            ImGui::EndMenu();
        }

        if (ImGui::BeginMenu("Help"))
        {
            ImGui::MenuItem("About");
            ImGui::EndMenu();
        }
        ImGui::EndMainMenuBar();
    }
}
```



図：9. メニューの画面イメージ

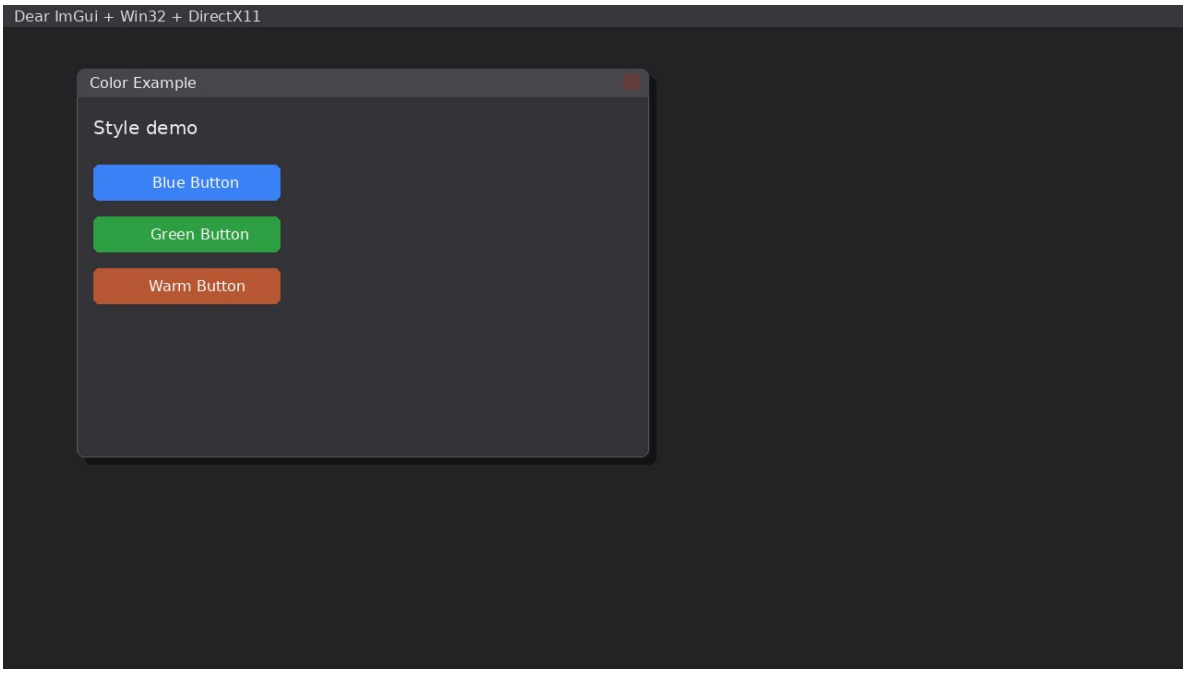
10. 色の替え方

- 一時的に色を変えるなら PushStyleColor / PopStyleColor が分かりやすい。
- ウィジェット単位の見た目変更をしたいときに便利。

```
void RenderUI()
{
    ImGui::Begin("Color Example");

    ImGui::PushStyleColor(ImGuiCol_Button,        ImVec4(0.23f, 0.51f, 0.96f, 1.0f));
    ImGui::PushStyleColor(ImGuiCol_ButtonHovered, ImVec4(0.33f, 0.61f, 1.00f, 1.0f));
    ImGui::PushStyleColor(ImGuiCol_ButtonActive,  ImVec4(0.16f, 0.42f, 0.86f, 1.0f));
    ImGui::Button("Blue Button", ImVec2(180, 0));
    ImGui::PopStyleColor(3);

    ImGui::End();
}
```



図：10. 色の替え方 の画面イメージ

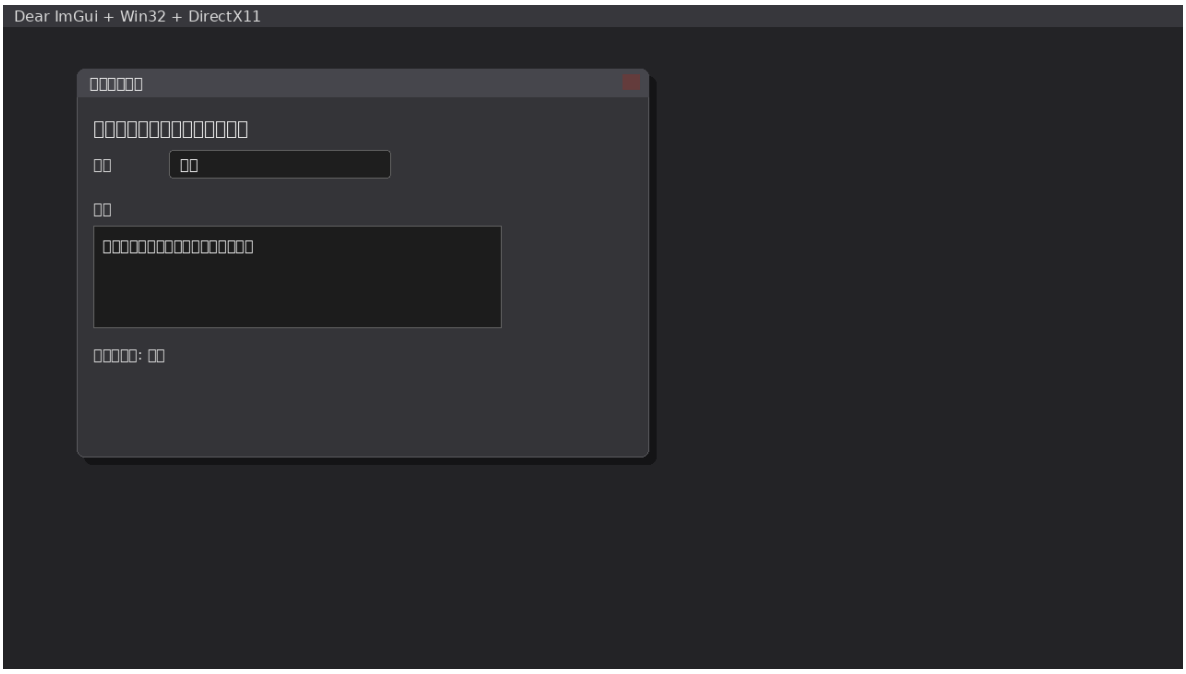
11. 日本語の方法

- 表示だけでなくラベル名も日本語にできる。
- 前提は UTF-8 保存、日本語フォント読み込み、GlyphRangesJapanese の指定。

```
void RenderUI()
{
    static char input_name[128] = "今井";
    static char memo[256] = "ここに日本語のメモを入力できます。";

    ImGui::SetNextWindowPos(ImVec2(40, 40), ImGuiCond_Once);
    ImGui::SetNextWindowSize(ImVec2(520, 320), ImGuiCond_Once);

    ImGui::Begin("日本語テスト");
    ImGui::Text("これは日本語表示の確認です。");
    ImGui::InputText("名前", input_name, IM_ARRAYSIZE(input_name));
    ImGui::InputTextMultiline("メモ", memo, IM_ARRAYSIZE(memo), ImVec2(440, 150));
    ImGui::Text("現在の名前: %s", input_name);
    ImGui::End();
}
```



図：11. 日本語の対応方法 の画面イメージ

12. Win32 API との行比較

以下は概算比較であり、実装流儀やエラーチェック量で増減する。 ただし傾向として、**UI 部分の記述量は ImGui の方がかなり少ない**。一方で Win32 + DirectX11 の土台コードは ImGui 利用時でも必要。

機能	ImGui の RenderUI 概算	Win32 API 単体 概算	差の理由
ウィンドウ内テキスト表示	5〜8 行	20〜35 行	CreateWindow, WM_PAINT, TextOut などが必要
単一行テキストボックス	8〜12 行	30〜50 行	EDIT コントロール生成・取得・配置が必要
複数行テキストボックス	10〜15 行	35〜60 行	スタイル指定やサイズ管理が増える
スライダー	6〜10 行	30〜55 行	TRACKBAR 作成、通知処理、値同期が必要
ボタン	6〜10 行	25〜45 行	BUTTON 作成、WM_COMMAND 分岐が必要
スクロール領域	10〜16 行	40〜80 行	子ウィンドウ・スクロール管理が重い
メニュー	12〜20 行	30〜70 行	HMENU 構築やコマンド ID 管理が必要
色変更	6〜12 行	20〜50 行	オーナードローや描画分岐が増える
日本語対応	main 側で行追加	比較的多め	フォント・文字コード・描画方式の管理がやや重

			い

務では『土台部分は一度作ったらほぼ固定』『RenderUI 側だけ頻繁に編集』という形にすると、ImGui の恩恵を受けやすい。

13. 最低限のチェックリスト

- imgui 本体 + backends の .cpp / .h をプロジェクトに追加したか
- include path に imgui ルートが入っているか
- リンクライブラリに d3d11 / dxgi / dxguid / dwmapi が入っているか
- main.cpp で ImGui_ImplWin32_Init / ImGui_ImplDX11_Init を呼んでいるか
- 日本語を使うなら AddFontFromFileTTF と GetGlyphRangesJapanese を入れたか
- ソスファイルを UTF-8 で保存したか

14. まとめ

Pleiades + Win32 + DirectX11 で Dear ImGui を使う場合、**基盤コードは多少必要**だが、ボタンや入力欄などの UI 実装は Win32 API 単体より大幅に短く、試作と変更がしやすい。

特に、**RenderUI** を **ui.cpp** に閉じ込めて編集する構成にすると、後からタスク管理アプリなどへ発展させやすい。